LP5 – Assignment 1

Implement multi-threaded client/server Process communication using RMI.

1. **AddClient.java**

```java
import java.rmi.*;

public class AddClient {
    public static void main(String args[]) {
        try {
            // Get reference to the remote object
            String addServerURL = "rmi://" + args[0] + "/AddServer";
            AddServerIntf addServerIntf = (AddServerIntf) Naming.lookup(addServerURL);

            System.out.println("The first number is: " + args[1]);
            double d1 = Double.parseDouble(args[1]);

            System.out.println("The second number is: " + args[2]);
            double d2 = Double.parseDouble(args[2]);

            // Invoke remote method to add numbers
            System.out.println("The sum is: " + addServerIntf.add(d1, d2));
        } catch (Exception e) {
            System.out.println("Exception: " + e);
        }
    }
}
```

2. **AddServer.java**

```java
import java.rmi.*;

public class AddServer {
    public static void main(String args[]) {
        try {
            // create remote object
            AddServerImpl addServerImpl = new AddServerImpl();

            // bind the remote object
            Naming.rebind("AddServer", addServerImpl);
        } catch (Exception e) {
            System.out.println("Exception: " + e);
        }
    }
}
```

3. **AddServerImpl.java**

```
import java.rmi.*;
import java.rmi.server.*;

//class that implements the remote interface
public class AddServerImpl extends UnicastRemoteObject implements AddServerIntf { //
constructor
    public AddServerImpl() throws RemoteException {
    }

    // implement method declared in the interface
    public double add(double d1, double d2) throws RemoteException {
        return d1 + d2;
    }
}
```

4. **AddServerIntf.java**

```
import java.rmi.*;

public interface AddServerIntf extends Remote {
    // method declaration
    double add(double d1, double d2) throws RemoteException;
}
```

LP5 – Assignment 2

Develop any distributed application using CORBA to demonstrate object brokering. (Calculator or String operations)

1. **ReverseClient.java**

```java
import ReverseModule.*;
import org.omg.CosNaming.*;
import org.omg.CosNaming.NamingContextPackage.*;
import org.omg.CORBA.*;
import java.io.*;

class ReverseClient {
    public static void main(String args[]) {
        Reverse ReverseImpl = null;
        try {
            // initialize the ORB
            org.omg.CORBA.ORB orb = org.omg.CORBA.ORB.init(args, null);
            org.omg.CORBA.Object objRef = orb.resolve_initial_references("NameService");

            NamingContextExt ncRef = NamingContextExtHelper.narrow(objRef);
            String name = "Reverse";

            ReverseImpl = ReverseHelper.narrow(ncRef.resolve_str(name));

            System.out.println("Enter String=");
            BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
            String str = br.readLine();
            String tempStr = ReverseImpl.reverse_string(str);

            System.out.println(tempStr);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

2. **ReverseImpl.java**

```java
import ReverseModule.ReversePOA;
import java.lang.String;

class ReverseImpl extends ReversePOA {
    ReverseImpl() {
        super();
        System.out.println("Reverse Object Created");
    }

    public String reverse_string(String name) {
        StringBuffer str = new StringBuffer(name);
        str.reverse();
        return (("Server Send " + str));
```

```
    }
}
```

3. **Server.java**

```
import ReverseModule.*;
import org.omg.CosNamig.*;
import org.omg.CosNaming.NamingContextPackage.*;
import org.omg.CORBA.*;
import org.omg.PortableServer.*;

class ReverseServer {
    public static void main(String[] args) {
        try {
            // initialize the ORB
            org.omg.CORBA.ORB orb = org.omg.CORBA.ORB.init(args, null);

            // initialize the BOA/POA
            POA rootPOA = POAHelper.narrow(orb.resolve_initial_references("RootPOA"));
            rootPOA.the_POAManager().activate(); // creating the calculator object
            ReverseImpl rvr = new ReverseImpl();

            // get the object reference from the servant class
            org.omg.CORBA.Object ref = rootPOA.servant_to_reference(rvr);
            System.out.println("Step1");

            Reverse h_ref = ReverseModule.ReverseHelper.narrow(ref);
            System.out.println("Step2");

            org.omg.CORBA.Object objRef = orb.resolve_initial_references("NameService");
            System.out.println("Step3");

            NamingContextExt ncRef = NamingContextExtHelper.narrow(objRef);
            System.out.println("Step4");
            String name = "Reverse";

            NameComponent path[] = ncRef.to_name(name);
            ncRef.rebind(path, h_ref);

            System.out.println("Reverse Server reading and waiting....");
            orb.run();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

LP5 – Assignment 3

Develop a distributed system, to find sum of `N` elements in an array by distributing `N/n` elements to `n` number of processors MPI or OpenMP. Demonstrate by displaying the intermediate sums calculated at different processors.

ArrSum.java

```
import mpi.MPI;
import java.util.Scanner;
import mpi.*;

public class ArrSum {
    public static void main(String[] args) throws Exception {
        MPI.Init(args);
        int rank = MPI.COMM_WORLD.Rank();
        int size = MPI.COMM_WORLD.Size();
        int unitsize = 5;
        int root = 0;
        int send_buffer[] = null;

        // 1 process is expected to handle 4 elements
        send_buffer = new int[unitsize * size];
        int recieve_buffer[] = new int[unitsize];
        int new_recieve_buffer[] = new int[size];

        // Set data for distribution
        if (rank == root) {
            int total_elements = unitsize * size;
            System.out.println("Enter " + total_elements + " elements");
            for (int i = 0; i < total_elements; i++) {
                System.out.println("Element " + i + "\t = " + i);
                send_buffer[i] = i;
            }
        }

        // Scatter data to processes
        MPI.COMM_WORLD.Scatter(send_buffer, 0, unitsize, MPI.INT, recieve_buffer, 0, unitsize, MPI.INT,
root);

        // Calculate sum at non root processes
        // Store result in first index of array
        for (int i = 1; i < unitsize; i++) {
            recieve_buffer[0] += recieve_buffer[i];
        }

        System.out.println("Intermediate sum at process " + rank + " is " + recieve_buffer[0]);

        // Gather data from processes
        MPI.COMM_WORLD.Gather(recieve_buffer, 0, 1, MPI.INT, new_recieve_buffer, 0, 1, MPI.INT, root);

        // Aggregate output from all non root processes
        if (rank == root) {
            int total_sum = 0;
            for (int i = 0; i < size; i++) {
                total_sum += new_recieve_buffer[i];
            }

            System.out.println("Final sum : " + total_sum);
        }
        MPI.Finalize();
    }
}
```

LP5 – Assignment 4

Implement Berkeley algorithm for clock synchronization

Client.py

```
# Python3 program imitating a client process

from timeit import default_timer as timer
from dateutil import parser
import threading
import datetime
import socket
import time

# client thread function used to send time at client side
def startSendingTime(slave_client):
    while True:
        # provide server with clock time at the client
        slave_client.send(str(datetime.datetime.now()).encode())

        print("Recent time sent successfully", end = "\n\n")
        time.sleep(5)


# client thread function used to receive synchronized time
def startReceivingTime(slave_client):
    while True:
        # receive data from the server
        Synchronized_time = parser.parse(slave_client.recv(1024).decode())

        print("Synchronized time at the client is: " + str(Synchronized_time), end =
"\n\n")

# function used to Synchronize client process time
def initiateSlaveClient(port = 8080):
    slave_client = socket.socket()

    # connect to the clock server on local computer
    slave_client.connect(('127.0.0.1', port))

    # start sending time to server
    print("Starting to receive time from server\n")
    send_time_thread = threading.Thread(target = startSendingTime, args = (slave_client, ))
    send_time_thread.start()

    # start receiving synchronized from server
    print("Starting to receiving " + "synchronized time from server\n")
    receive_time_thread = threading.Thread(target = startReceivingTime, args =
(slave_client, ))
    receive_time_thread.start()

# Driver function
if __name__ == '__main__':
```

```
    # initialize the Slave / Client
    initiateSlaveClient(port = 8080)
```

Server.py

```
# Python3 program imitating a clock server

from dateutil import parser
import threading
import datetime
import socket
import time

# datastructure used to store client address and clock data

client_data = {}

# nested thread function used to receive clock time from a connected client
def startReceivingClockTime(connector, address):
    while True:
        # receive clock time
        clock_time_string = connector.recv(1024).decode()
        clock_time = parser.parse(clock_time_string)
        clock_time_diff = datetime.datetime.now() - clock_time

        client_data[address] = {"clock_time": clock_time,
                                "time_difference": clock_time_diff,
                                "connector": connector}

        print("Client Data updated with: "+ str(address), end = "\n\n")

        time.sleep(5)

# master thread function used to open portal for accepting clients over given port
def startConnecting(master_server):
    # fetch clock time at slaves / clients
    while True:
        # accepting a client / slave clock client
        master_slave_connector, addr = master_server.accept()
        slave_address = str(addr[0]) + ":" + str(addr[1])

        print(slave_address + " got connected successfully")

        current_thread = threading.Thread(target = startReceivingClockTime,
                                          args = (master_slave_connector, slave_address, ))
        current_thread.start()

# subroutine function used to fetch average clock difference
def getAverageClockDiff():
    time_difference_list = list(client['time_difference'])

    for client_addr, client in client_data.items():
    sum_of_clock_difference = sum(time_difference_list, datetime.timedelta(0, 0))
    average_clock_difference = sum_of_clock_difference / len(client_data)

    return average_clock_difference
```

```python
# master sync thread function used to generate cycles of clock synchronization in the network
def synchronizeAllClocks():
    while True:

        print("New synchronization cycle started.")
        print("Number of clients to be synchronized: " + str(len(client_data)))

        if len(client_data) > 0:
            average_clock_difference = getAverageClockDiff()
            for client_addr, client in client_data.items():
                try:
                   synchronized_time = datetime.datetime.now() + average_clock_difference
                   client['connector'].send(str(synchronized_time).encode())

                except Exception as e:
                    print("Something went wrong while " +
                        "sending synchronized time " +
                        "through " + str(client_addr))

        else:
            print("No client data." + " Synchronization not applicable.")

        print("\n\n")
        time.sleep(5)

# function used to initiate the Clock Server / Master Node
def initiateClockServer(port = 8080):
    master_server = socket.socket()
    master_server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    print("Socket at master node created successfully\n")
    master_server.bind(('', port))

    # Start listening to requests
    master_server.listen(10)
    print("Clock server started...\n")

    # start making connections
    print("Starting to make connections...\n")
    master_thread = threading.Thread(target = startConnecting, args = (master_server, ))
    master_thread.start()     # start synchronization
    print("Starting synchronization parallelly...\n")
    sync_thread = threading.Thread(target = synchronizeAllClocks, args = ())
    sync_thread.start()

    #Driver function
    if __name__ == '__main__':
        # Trigger the Clock Server
        initiateClockServer(port = 8080)
```

LP5 – Assignment 5

Implement token ring based mutual exclusion algorithm

TokenRingAlgorithm.java

```
import java.io.*;

public class TokenRingAlgorithm {
    public static void main(String args[]) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        // Input number of nodes in the ring
        System.out.print("Enter the number of nodes in the ring: ");
        int numNodes = Integer.parseInt(br.readLine());

        // Input sender and receiver node IDs (0-indexed)
        System.out.print("Enter sender node id (0 to " + (numNodes - 1) + "): ");
        int sender = Integer.parseInt(br.readLine());
        System.out.print("Enter receiver node id (0 to " + (numNodes - 1) + "): ");
        int receiver = Integer.parseInt(br.readLine());

        // Input the data to send
        System.out.print("Enter the data to send: ");
        String data = br.readLine();

        System.out.println("\nToken passing simulation:");

        // Simulate token passing from node 0 until the sender is reached
        int token = 0;
        while (token != sender) {
            System.out.println("Token at node " + token);
            token = (token + 1) % numNodes;
        }

        System.out.println("Token reached sender node " + sender);
        System.out.println("Sender node " + sender + " is sending data to receiver node "
+ receiver);
        System.out.println("Data sent: " + data);

        // Continue passing token from sender until it reaches the receiver
        token = sender;
        while (token != receiver) {
            int next = (token + 1) % numNodes;
            System.out.println("Token passing from node " + token + " to node " + next);
            token = next;
        }

        System.out.println("Token reached receiver node " + receiver);
        System.out.println("Receiver node " + receiver + " received the data: " + data);
    }
}
```

Output:

```
/m/d/c/4/B/L/A/5

rudrkadam@Rudra-Legion5 /m/d/c/4/B/L/A/5> javac TokenRingAlgorithm.java
rudrkadam@Rudra-Legion5 /m/d/c/4/B/L/A/5> java TokenRingAlgorithm
Enter the number of nodes in the ring: 5
Enter sender node id (0 to 4): 2
Enter receiver node id (0 to 4): 4
Enter the data to send: Hello CORBA

Token passing simulation:
Token at node 0
Token at node 1
Token reached sender node 2
Sender node 2 is sending data to receiver node 4
Data sent: Hello CORBA
Token passing from node 2 to node 3
Token passing from node 3 to node 4
Token reached receiver node 4
Receiver node 4 received the data: Hello CORBA
rudrkadam@Rudra-Legion5 /m/d/c/4/B/L/A/5>
```

LP5 – Assignment 6

Implement Bully and Ring algorithm for leader election

Bully.java

```
import java.util.*;

public class Bully {

    int coordinator;
    int max_processes;
    boolean processes[];

    public Bully(int max) {
        max_processes = max;
        processes = new boolean[max_processes];
        coordinator = max;

        System.out.println("Creating processes..");
        for (int i = 0; i < max; i++) {
            processes[i] = true;
            System.out.println("P" + (i + 1) + " created");
        }

        System.out.println("Process P" + coordinator + " is the coordinator");
    }

    void displayProcesses() {

        for (int i = 0; i < max_processes; i++) {
            if (processes[i]) {
                System.out.println("P" + (i + 1) + " is up");

            } else
                System.out.println("P" + (i + 1) + " is down");
        }

        System.out.println("Process P" + coordinator + " is the coordinator");
    }

    void upProcess(int process_id) {
        if (!processes[process_id - 1]) {
            processes[process_id - 1] = true;
            System.out.println("Process " + process_id + " is now up.");
        } else
            System.out.println("Process " + process_id + " is already up.");
    }

    void downProcess(int process_id) {
        if (!processes[process_id - 1]) {
            System.out.println("Process " + process_id + " is already down.");
        } else {
            processes[process_id - 1] = false;
            System.out.println("Process " + process_id + " is down.");
        }
    }

    void runElection(int process_id) {
        coordinator = process_id;
        boolean keepGoing = true;
```

```java
        for (int i = process_id; i < max_processes && keepGoing; i++) {
            System.out.println("Election message sent from process " + process_id + " to process " + (i
+ 1));
            if (processes[i]) {
                keepGoing = false;
                runElection(i + 1);
            }
        }
    }

    public static void main(String args[]) {
        Bully bully = null;
        int max_processes = 0, process_id = 0;
        int choice = 0;
        Scanner sc = new Scanner(System.in);

        while (true) {
            System.out.println("Bully Algorithm");
            System.out.println("1. Create processes");
            System.out.println("2. Display processes");
            System.out.println("3. Up a process");
            System.out.println("4. Down a process");
            System.out.println("5. Run election algorithm");
            System.out.println("6. Exit Program");
            System.out.print("Enter your choice:- ");
            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    System.out.print("Enter the number of processes:- ");
                    max_processes = sc.nextInt();
                    bully = new Bully(max_processes);
                    break;
                case 2:
                    bully.displayProcesses();
                    break;
                case 3:
                    System.out.print("Enter the process number to up:- ");
                    process_id = sc.nextInt();
                    bully.upProcess(process_id);
                    break;
                case 4:
                    System.out.print("Enter the process number to down:- ");
                    process_id = sc.nextInt();
                    bully.downProcess(process_id);
                    break;
                case 5:
                    System.out.print("Enter the process number which will perform election:- ");
                    process_id = sc.nextInt();
                    bully.runElection(process_id);
                    bully.displayProcesses();
                    break;
                case 6:
                    System.exit(0);
                    break;
                default:
                    System.out.println("Error in choice. Please try again.");
                    break;
            }
        }
    }
}
```

Ring.java

```java
import java.util.*;

public class Ring {

    int max_processes;
    int coordinator;
    boolean processes[];
    ArrayList<Integer> pid;

    public Ring(int max) {
        coordinator = max;
        max_processes = max;
        pid = new ArrayList<Integer>();
        processes = new boolean[max];

        for (int i = 0; i < max; i++) {
            processes[i] = true;
            System.out.println("P" + (i + 1) + " created.");
        }

        System.out.println("P" + (coordinator) + " is the coordinator");
    }

    void displayProcesses() {
        for (int i = 0; i < max_processes; i++) {
            if (processes[i]) System.out.println("P" + (i + 1) + " is up.");
            else System.out.println("P" + (i + 1) + " is down.");
        }

        System.out.println("P" + (coordinator) + " is the coordinator");
    }

    void upProcess(int process_id) {
        if (!processes[process_id - 1]) {
            processes[process_id - 1] = true;
            System.out.println("Process P" + (process_id) + " is up.");
        } else System.out.println("Process P" + (process_id) + " is already up.");
    }

    void downProcess(int process_id) {
        if (!processes[process_id - 1]) {
            System.out.println("Process P" + (process_id) + " is already down.");
        } else {
            processes[process_id - 1] = false;
            System.out.println("Process P" + (process_id) + " is down.");
        }
    }

    void displayArrayList(ArrayList<Integer> pid) {
        System.out.print("[ ");
        for (Integer x : pid) System.out.print(x + " ");

        System.out.print(" ]\n");
    }

    void initElection(int process_id) {
        if (processes[process_id - 1]) {
            pid.add(process_id);
```

```java
            int temp = process_id;

            System.out.print("Process P" + process_id + " sending the following list:- ");
            displayArrayList(pid);

            while (temp != process_id - 1) {
                if (processes[temp]) {
                    pid.add(temp + 1);
                    System.out.print("Process P" + (temp + 1) + " sending the following list:- ");
                    displayArrayList(pid);
                }
                temp = (temp + 1) % max_processes;
            }

            coordinator = Collections.max(pid);
            System.out.println("Process P" + process_id + " has declared P" + coordinator + " as the
coordinator");
            pid.clear();
        }
    }

    public static void main(String args[]) {
        Ring ring = null;

        int max_processes = 0, process_id = 0;
        int choice = 0;
        Scanner sc = new Scanner(System.in);

        while (true) {
            System.out.println("Ring Algorithm");
            System.out.println("1. Create processes");
            System.out.println("2. Display processes");
            System.out.println("3. Up a process");
            System.out.println("4. Down a process");
            System.out.println("5. Run election algorithm");
            System.out.println("6. Exit Program");
            System.out.print("Enter your choice:- ");
            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    System.out.print("Enter the total number of processes:- ");
                    max_processes = sc.nextInt();
                    ring = new Ring(max_processes);
                    break;
                case 2:
                    ring.displayProcesses();
                    break;
                case 3:
                    System.out.print("Enter the process to up:- ");
                    process_id = sc.nextInt();
                    ring.upProcess(process_id);
                    break;
                case 4:
                    System.out.print("Enter the process to down:- ");
                    process_id = sc.nextInt();
                    ring.downProcess(process_id);
                    break;
                case 5:
                    System.out.print("Enter the process which will initiate election:- ");
                    process_id = sc.nextInt();
                    ring.initElection(process_id);
                    break;
                case 6:
                    System.exit(0);
```

```
                    break;
                default:
                    System.out.println("Error in choice. Please try again.");
                    break;
            }
        }
    }
}
```

Output:

1. **Bully Algorithm:**

```
/m/d/c/4/B/L/A/6

rudrkadam@Rudra-Legion5 /m/d/c/4/B/L/A/6> javac Bully.java
rudrkadam@Rudra-Legion5 /m/d/c/4/B/L/A/6> java Bully
Bully Algorithm
1. Create processes
2. Display processes
3. Up a process
4. Down a process
5. Run election algorithm
6. Exit Program
Enter your choice:- 1
Enter the number of processes:- 4
Creating processes..
P1 created
P2 created
P3 created
P4 created
Process P4 is the coordinator
Bully Algorithm
1. Create processes
2. Display processes
3. Up a process
4. Down a process
5. Run election algorithm
6. Exit Program
Enter your choice:- 2
P1 is up
P2 is up
P3 is up
P4 is up
Process P4 is the coordinator
Bully Algorithm
1. Create processes
2. Display processes
3. Up a process
4. Down a process
5. Run election algorithm
6. Exit Program
Enter your choice:- 4
Enter the process number to down:- 2
```

/m/d/c/4/B/L/A/6

```
Process 2 is down.
Bully Algorithm
1. Create processes
2. Display processes
3. Up a process
4. Down a process
5. Run election algorithm
6. Exit Program
Enter your choice:- 2
P1 is up
P2 is down
P3 is up
P4 is up
Process P4 is the coordinator
Bully Algorithm
1. Create processes
2. Display processes
3. Up a process
4. Down a process
5. Run election algorithm
6. Exit Program
Enter your choice:- 5
Enter the process number which will perform election:- 3
Election message sent from process 3 to process 4
P1 is up
P2 is down
P3 is up
P4 is up
Process P4 is the coordinator
Bully Algorithm
1. Create processes
2. Display processes
3. Up a process
4. Down a process
5. Run election algorithm
6. Exit Program
Enter your choice:- 6
rudrkadam@Rudra-Legion5 /m/d/c/4/B/L/A/6>
```

2. **Ring Algorithm:**

```
/m/d/c/4/B/L/A/6

rudrkadam@Rudra-Legion5 /m/d/c/4/B/L/A/6> javac Ring.java
rudrkadam@Rudra-Legion5 /m/d/c/4/B/L/A/6> java Ring
Ring Algorithm
1. Create processes
2. Display processes
3. Up a process
4. Down a process
5. Run election algorithm
6. Exit Program
Enter your choice:- 1
Enter the total number of processes:- 4
P1 created.
P2 created.
P3 created.
P4 created.
P4 is the coordinator
Ring Algorithm
1. Create processes
2. Display processes
3. Up a process
4. Down a process
5. Run election algorithm
6. Exit Program
Enter your choice:- 2
P1 is up.
P2 is up.
P3 is up.
P4 is up.
P4 is the coordinator
Ring Algorithm
1. Create processes
2. Display processes
3. Up a process
4. Down a process
5. Run election algorithm
6. Exit Program
Enter your choice:- 4
Enter the process to down:- 2
Process P2 is down.
```

/m/d/c/4/B/L/A/6

```
Ring Algorithm
1. Create processes
2. Display processes
3. Up a process
4. Down a process
5. Run election algorithm
6. Exit Program
Enter your choice:- 2
P1 is up.
P2 is down.
P3 is up.
P4 is up.
P4 is the coordinator
Ring Algorithm
1. Create processes
2. Display processes
3. Up a process
4. Down a process
5. Run election algorithm
6. Exit Program
Enter your choice:- 5
Enter the process which will initiate election:- 3
Process P3 sending the following list:- [ 3  ]
Process P4 sending the following list:- [ 3 4  ]
Process P1 sending the following list:- [ 3 4 1  ]
Process P3 has declared P4 as the coordinator
Ring Algorithm
1. Create processes
2. Display processes
3. Up a process
4. Down a process
5. Run election algorithm
6. Exit Program
Enter your choice:- 6
rudrkadam@Rudra-Legion5 /m/d/c/4/B/L/A/6>
```

LP5 – Assignment 7

Create a simple web service and write any distributed application to consume the web service

SimpleInterest.java

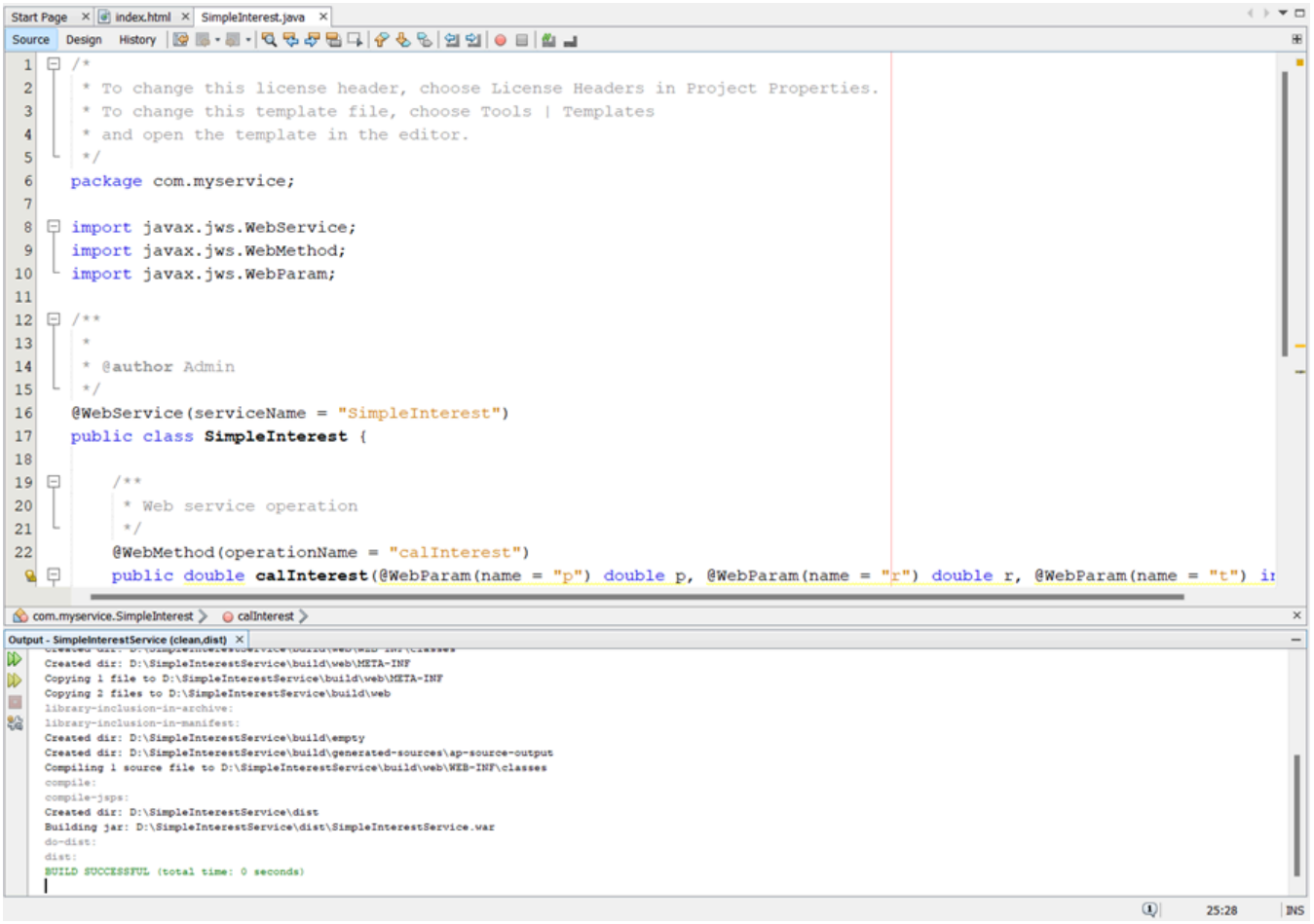
```java
package com.myservice;

import javax.jws.WebService;
import javax.jws.WebMethod;
import javax.jws.WebParam;

@WebService(serviceName = "SimpleInterest")
public class SimpleInterest {
    // Web service operation
    @WebMethod(operationName = "calInterest")
    public double calInterest(@WebParam(name = "p") double p,
                              @WebParam(name = "r") double r,
                              @WebParam(name = "t") int t) {

        return (p*r*t)/100;
    }
}
```

Output:



Start Page | index.html | SimpleInterest.java

Source | Design | History

```java
/*
 * To change this license header, choose License Headers in Project Properties.
 * To change this template file, choose Tools | Templates
 * and open the template in the editor.
 */
package com.myservice;

import javax.jws.WebService;
import javax.jws.WebMethod;
import javax.jws.WebParam;

/**
 *
 * @author Admin
 */
@WebService(serviceName = "SimpleInterest")
public class SimpleInterest {

    /**
     * Web service operation
     */
    @WebMethod(operationName = "calInterest")
    public double calInterest(@WebParam(name = "p") double p, @WebParam(name = "r") double r, @WebParam(name = "t") i
```

com.myservice.SimpleInterest > calInterest

Output

SimpleInterestService (run-deploy) | Java DB Database Process | GlassFish Server 4.1.1

```
Info:   Webservice Endpoint deployed RegistrationPortTypeRPCPortImpl
 listening at address at http://LAPTOP-O79ADNNF:8080/__wstx-services/RegistrationPortTypeRPC11.
Info:   Webservice Endpoint deployed RegistrationPortTypeImpl
 listening at address at http://LAPTOP-O79ADNNF:8080/__wstx-services/RegistrationPortTypeRPC.
Info:   Webservice Endpoint deployed ParticipantPortTypePortImpl
 listening at address at http://LAPTOP-O79ADNNF:8080/__wstx-services/ParticipantPortType11.
Info:   Webservice Endpoint deployed CoordinatorPortTypePortImpl
 listening at address at http://LAPTOP-O79ADNNF:8080/__wstx-services/CoordinatorPortType11.
Info:   Loading application [wstx-services] at [/__wstx-services]
Info:   WS-TX Services successfully started.
Info:   Loading application [SimpleInterestService] at [/SimpleInterestService]
Info:   SimpleInterestService was successfully deployed in 2,907 milliseconds.
```

25:28 INS

localhost:8080/SimpleInterestService/SimpleInterest?Tester

Import favorites | Amazon.co.uk - Onl... | Express VPN | McAfee Security | LastPass password... | Gmail | YouTube

SimpleInterest Web Service Tester

This form will allow you to test your web service implementation (WSDL File)

To invoke an operation, fill the method parameter(s) input boxes and click on the button labeled with the method name.

Methods :

public abstract double com.myservice.SimpleInterest.calInterest(double,double,int)

calInterest (, ,)

Method parameter(s)

Type	Value
double	1000
double	2
int	1

Method returned

double : "**20.0**"

SOAP Request

```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Header/>
    <S:Body>
        <ns2:calInterest xmlns:ns2="http://myservice.com/">
            <p>1000.0</p>
            <r>2.0</r>
            <t>1</t>
        </ns2:calInterest>
    </S:Body>
</S:Envelope>
```

SOAP Response

```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Header/>
    <S:Body>
        <ns2:calInterestResponse xmlns:ns2="http://myservice.com/">
            <return>20.0</return>
        </ns2:calInterestResponse>
    </S:Body>
```